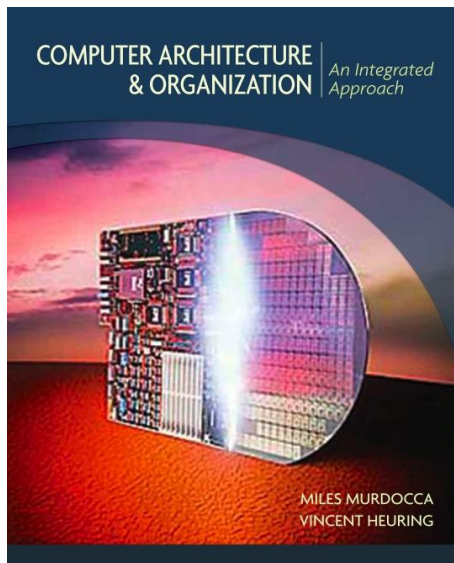


Organización de las Computadoras

Carolina Chaves Garcia



Programación ESP32-C3

Assembler puro

Assembler puro

¿Qué es cada archivo?

ARCHIVO	¿QUÉ ES?	PARA QUÉ SIRVE
.o	Código ensamblado, aún sin direcciones fijas	Entrada para el linker
.elf	Ejecutable con secciones y símbolos organizados	Para depurar o convertir a .bin
.bin	Binario crudo listo para flashear	Lo flashea esptool.py
.ld	Script del linker, dice dónde va cada cosa	Guía para crear el .elf correctamente

Assembler puro

Salida de la terminal

```
riscv32-esp-elf-as -march=rv32imc -mabi=ilp32 -g -o main.o main.S
riscv32-esp-elf-gcc -T linker.ld -nostartfiles -nostdlib -Wl,--gc-sections -Wl,-Map=main.map -Wl,--print-memory-usage -Wl,--cref -o main.elf main.o
```

- En la primera línea estamos usando el assembler (as) para convertir nuestro archivo
 - main.S (Assembly) en un archivo objeto main.o.
 - march=rv32imc: Target architecture = RISC-V 32-bit, con extensiones Integer (I), Multiplication/Division (M), y Compressed instructions (C).
 - mabi=ilp32: ABI = Integers, longs y punteros son de 32 bits.
 - g: Incluye información de depuración (usamos gdb).
 - o main.o: Salida = archivo objeto.
 -
- En la segunda línea se usa gcc como frontend para pasarle todo al linker (ld). Es decir, estamos enlazando main.o para generar un ejecutable main.elf.

Memory region	Used Size	Region Size	%age Used
iram:	48 B	128 KB	0.04%
dram:	16 KB	128 KB	12.50%

Usando C

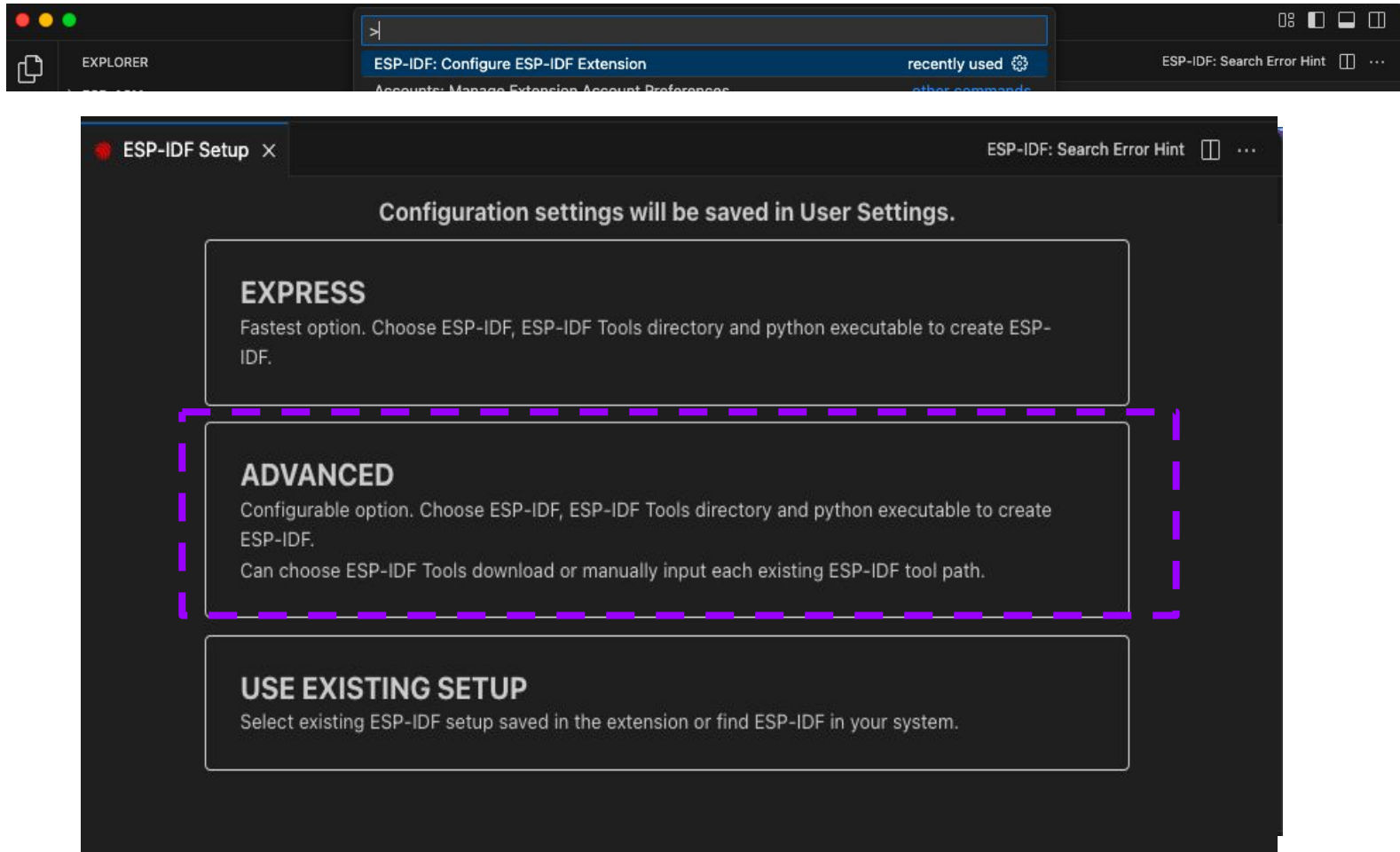
Características generales

- Assembler puro nos permite tener el control total sobre
 - Los argumentos que pasamos a una función.
 - El manejo de los registros
 - La limpieza de la pila
- Aunque es **flexible** nos puede generar un **caos** si no estamos seguros de las direcciones y manejo de pila, especialmente en el flasheo y cuando interactuamos con códigos de terceros (librerías).
- La convención de llamadas en C es un estándar que define exactamente como debe hacerse.
- Esto nos permite hacer una gestión automática de la pila.
- Permite que nuestro código pueda hacer parte de un ecosistema más grande sin quitarnos el control que tenemos en assembler.

Usando C

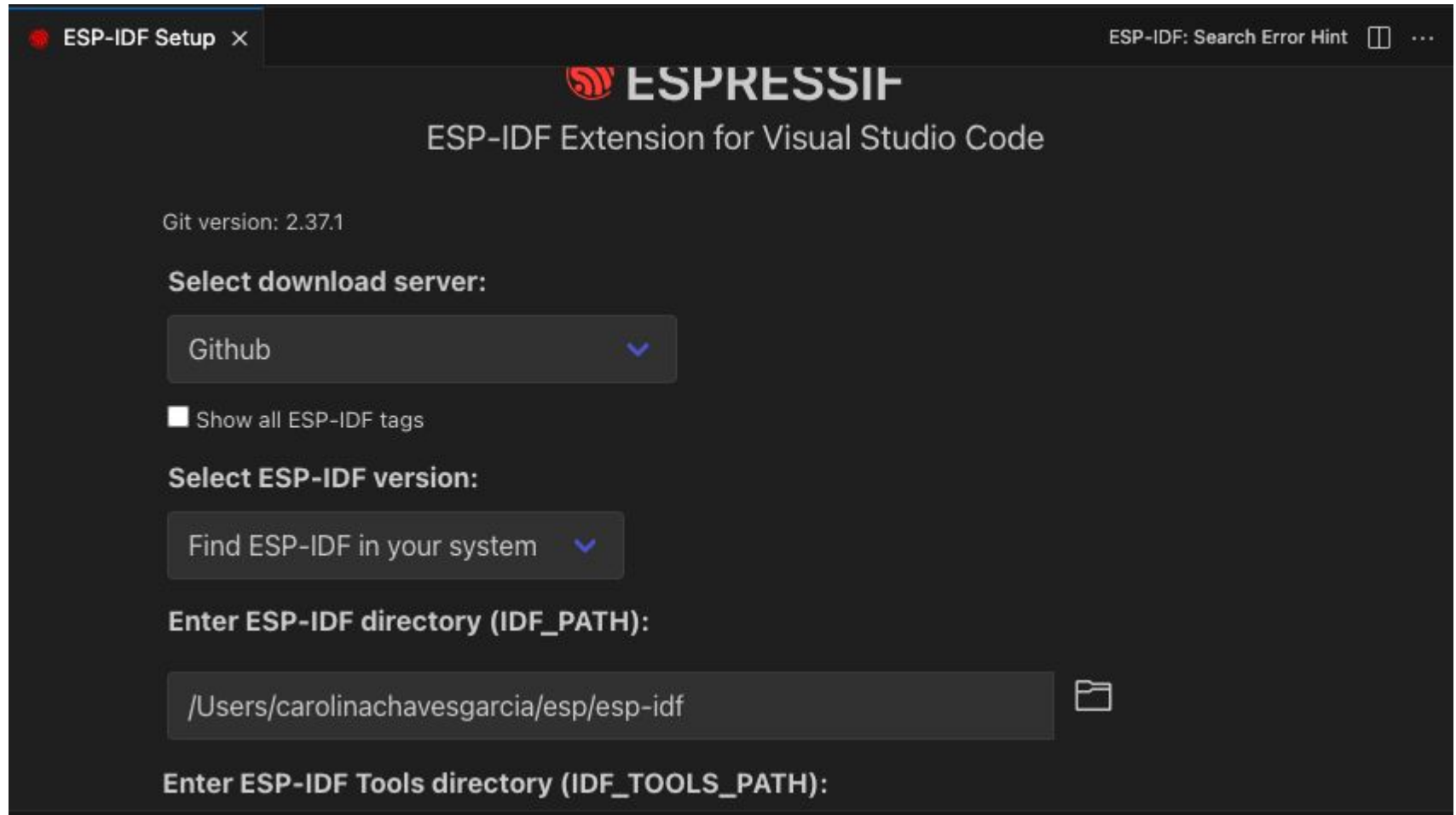
Crear un proyecto

- Oprime F1 y busca “ESP-IDF: Configure ESP-IDF extension”



Usando C

Crear un proyecto



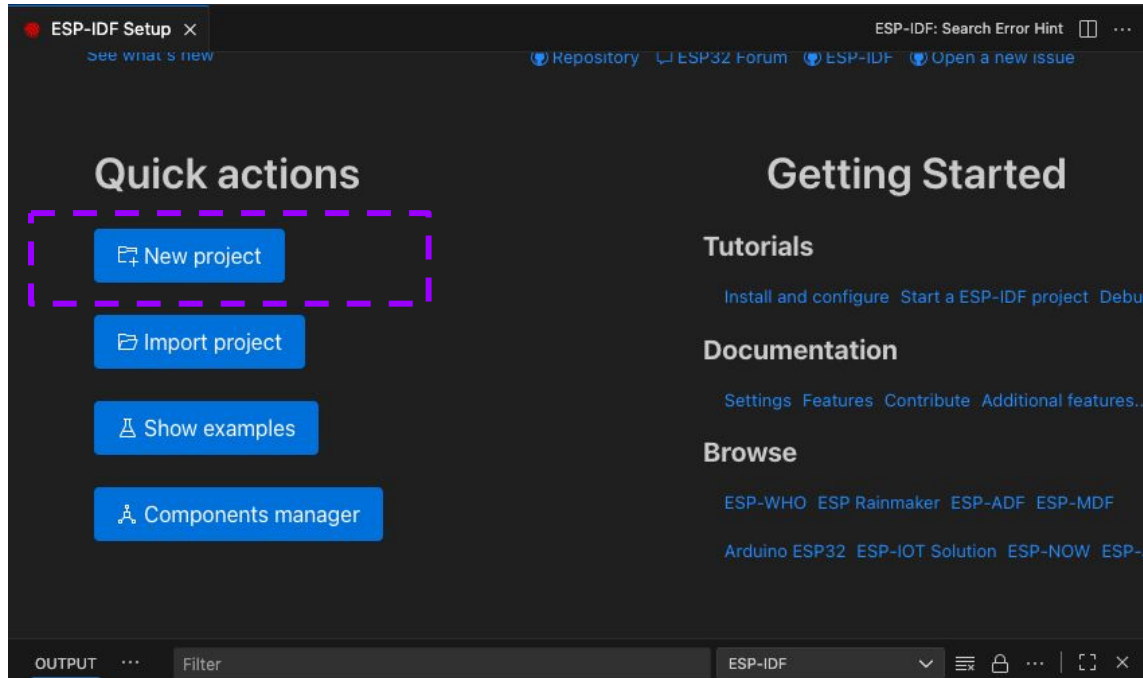
The screenshot shows the 'ESP-IDF Setup' window in Visual Studio Code. The window has a dark theme. At the top, there is a title bar with 'ESP-IDF Setup' and a close button. Below the title bar, the Espressif logo is displayed, followed by the text 'ESP-IDF Extension for Visual Studio Code'. The main content area contains the following elements:

- 'Git version: 2.37.1' text.
- 'Select download server:' label followed by a dropdown menu showing 'Github'.
- A checkbox labeled 'Show all ESP-IDF tags' which is currently unchecked.
- 'Select ESP-IDF version:' label followed by a dropdown menu showing 'Find ESP-IDF in your system'.
- 'Enter ESP-IDF directory (IDF_PATH):' label followed by a text input field containing '/Users/carolinachavesgarcia/esp/esp-idf' and a folder icon button to the right.
- 'Enter ESP-IDF Tools directory (IDF_TOOLS_PATH):' label.

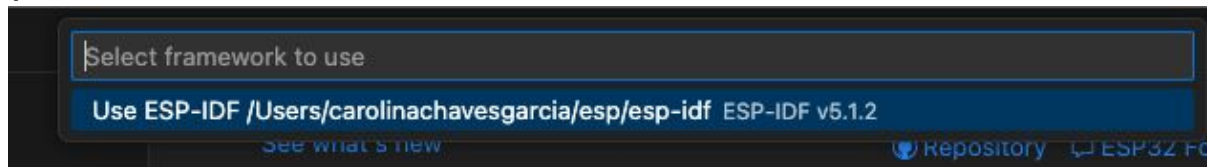
Usando C

Crear un proyecto

- Una vez finaliza la instalación tenemos



- Debemos escoger donde hicimos la pre-instalacion del ESP-IDF. Por defecto aparece



Usando C

Crear proyecto

Project Name

Ejemplo

Enter Project directory

/Users/carolinachavesgarcia /Ejemplo 

Choose ESP-IDF Target (IDF_TARGET)

esp32c3 ▼

Choose ESP-IDF Board

ESP32-C3 chip (via builtin USB-JTAG) ▼

Choose serial port

/dev/tty.usbserial-1410 ▼

Usando C

Crear un proyecto

- Podemos escoger de plantilla algún ejemplo. En esta caso usaremos “hello-world”

New Project

ESP-IDF ▼

Search Template By Name

get-started

- blink
- hello_world**
- sample_project

bluetooth

- ble
- ble_throughput
- throughput_client
- throughput_server
- ble_ancs

Supported Targets

ESP32	ESP32-C2	ESP32-C3	ESP32-C6	ESP32-H2	ESP32-S2	ESP32-S3

Hello World Example

Starts a FreeRTOS task to print "Hello World".

(See the README.md file in the upper level 'examples' directory for more information about examples.)

How to use example

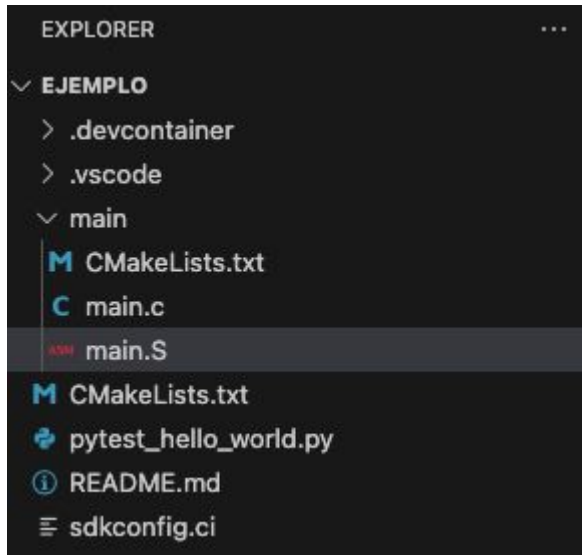
Follow detailed instructions provided specifically for this example.

Select the instructions depending on Espressif chip installed on your development board:

Usando C

Crear un proyecto

- Creamos un archivo assembler (.S) dentro de la carpeta main.



```

main.S
/*
 * Ejemplo 1: Carga el valor 16 en a2 y entra en bucle
 */

.section .text
.align 4
.global assembly_function # Hacer visible para C

assembly_function:
    li a2, 16           # Carga inmediata de 16 en registro a2
    la a3, test_value   # Cargar dirección de test_value
    sw a2, 0(a3)         # Guardar valor en memoria

    # Bucle infinito seguro para ESP32-C3 (tu código exacto)
1:
    wfi                 # Instrucción Wait For Interrupt
    j 1b                # Salto al label anterior
  
```

```

/*
 * Ejemplo 1: Carga el valor 16 en a2 y entra en bucle
 */

.section .data
test_value: .word 0x3FC80010

.section .text
.align 4
.global assembly_function # Hacer visible para C

assembly_function:
    li a2, 16           # Carga inmediata de 16 en registro a2
    la a3, test_value   # Cargar dirección de test_value
    sw a2, 0(a3)         # Guardar valor en memoria

    ret
  
```

Usando C

Crear un proyecto

- Modificar el main.c dentro de la carpeta main.

```
#include <stdio.h>
#include <inttypes.h>
#include "sdkconfig.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_chip_info.h"
#include "esp_flash.h"

// Declarar función externa de assembler
extern void assembly_function(void);

void app_main(void)
{
    printf("Iniciando programa con assembler...\n");

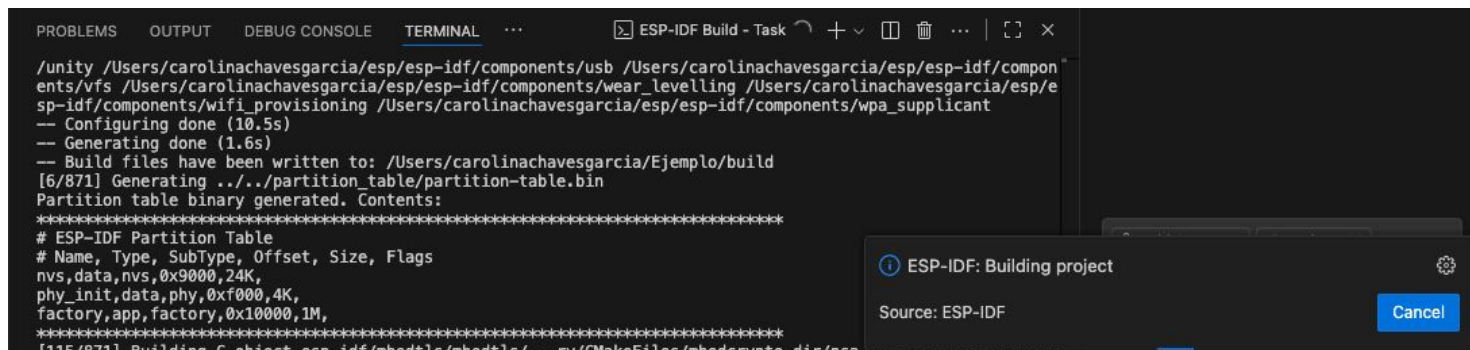
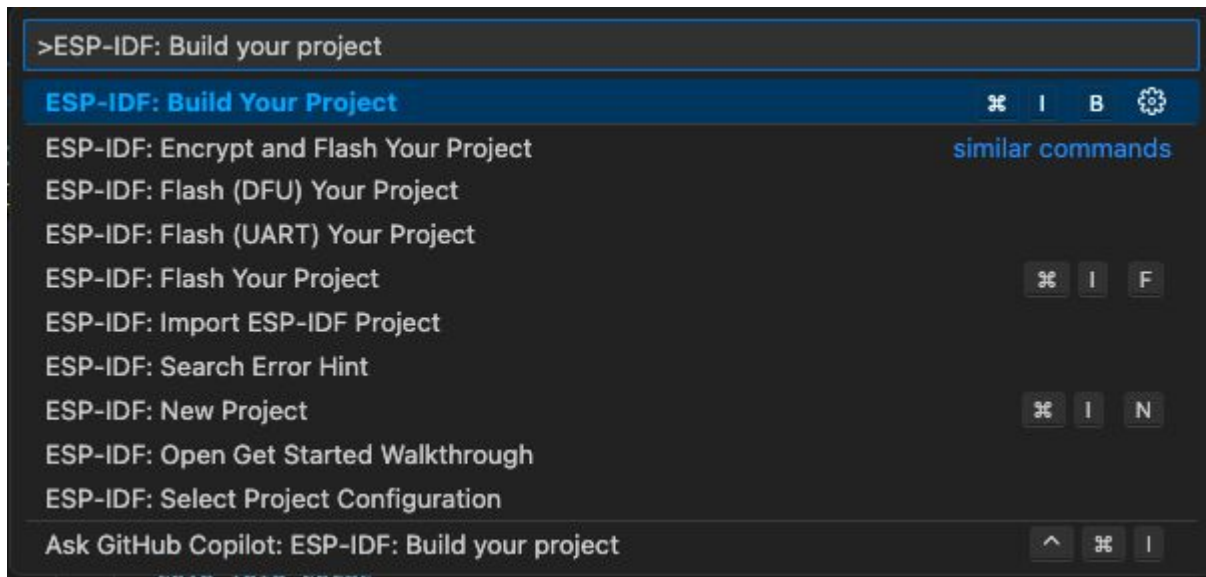
    // Llamar a la función en assembler
    assembly_function();

    printf("Ensamblador ejecutado. Entrando en bucle principal...\n");
}
```

Usando C

Compilación y flasheo

- F1 y buscamos ESP-IDF: Build your project



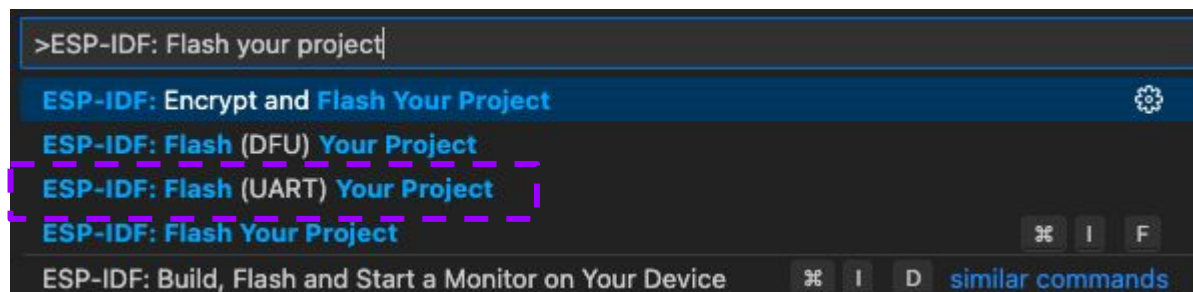
Arquitectura RISC-V del ESP32-C3

Registros

- Una vez finaliza, buscamos “ESP-IDF: Flash your project”

```
* Executing task: /Users/carolinachavesgarcia/.espressif/python_env/idf5.1_py3.13_env/bin/python /Users/carolinachavesgarcia/esp/esp-idf/tools/idf_size.py /Users/carolinachavesgarcia/Ejemplo/build/Ejemplo.map

pyenv shell 3.10.8
Total sizes:
Used stat D/IRAM: 50556 bytes ( 270740 remain, 15.7% used)
  .data size: 4388 bytes
  .bss size: 3656 bytes
  .text size: 42512 bytes
Used Flash size : 122576 bytes
  .text: 87872 bytes
  .rodata: 34448 bytes
Total image size: 169476 bytes (.bin may be padded larger)
```



Arquitectura RISC-V del ESP32-C3

Registros

```
idf.py -p PORT flash
or
python -m esptool --chip esp32c3 -b 460800 --before default_reset --after hard_reset --port /dev/tty.
usbserial-1410 write_flash --flash_mode dio --flash_size 2MB --flash_freq 80m 0x0 bootloader/bootloader.
bin 0x10000 Ejemplo.bin 0x8000 partition_table/partition-table.bin
or from the "/Users/carolinachavesgarcia/Ejemplo/build" directory
python -m esptool --chip esp32c3 -b 460800 --before default_reset --after hard_reset write_flash
"@flash_args"
[/Build]
[Flash]
Flash Done ⚡
Flash has finished. You can monitor your device with 'ESP-IDF: Monitor command'
```

Add Context... main.S X

Add context (#), extensions (@), command

Ask GPT-4.1

- Una vez finaliza, buscamos “ESP-IDF: Monitor Device”

```
on_env/idf5.1_py3.13_env/bin/python' '/Users/carolinachavesgarcia/esp/esp-idf/tools/idf_mon
itor.py' -p /dev/tty.usbserial-1410 -b 115200 --toolchain-prefix riscv32-esp-elf- --make ''
/Users/carolinachavesgarcia/.espressif/python_env/idf5.1_py3.13_env/bin/python' '/Users/car
olinachavesgarcia/esp/esp-idf/tools/idf.py' --target esp32c3 '/Users/carolinachavesgarcia/
Ejemplo/build/Ejemplo.elf'
Bucle principal C - ejecutando...
Bucle principal C - ejecutando...
Bucle principal C - ejecutando...
Bucle principal C - ejecutando...
Bucle principal C - ejecutando...
Bucle principal C - ejecutando...
Bucle principal C - ejecutando...
Bucle principal C - ejecutando...
Bucle principal C - ejecutando...
Bucle principal C - ejecutando...
```


Guía Práctica Programació nº 5 - RiscV

Guía Práctica Programació nº 5 - RiscV

Implementa los punto 3 y 4 usando llamadas desde C

Punto 3:

- a. Implementa el ejercicio 7 de la guia “Guía Práctica Programacion nº 1 – ASM” en el ESP32-C3.
- b. Implementa el ejercicio 9 de la guia “Guía Práctica Programacion nº 1 – ASM” en el ESP32-C3.
- c. Implementa el ejercicio 1 (solo la suma) de la guia “Guía Práctica Programacion nº 2 – ASM” en el ESP32-C3.

Punto 4:

- d. Ejercicio 1:
 - i. Simula una operación de copia de "inputs" de un sensor a un buffer de entrada para inferencia.
 - ii. Se te da un arreglo de 8 números enteros simulando datos de entrada de sensores. Se deben transferir estos datos a un segundo arreglo (buffer de procesamiento). Hazlo utilizando un bucle y registros temporales.

Datos:

.data

input: .word 10, 20, 30, 40, 50, 60, 70, 80

buffer: .space 32

Guía Práctica Programació nº 5 - RiscV

Implementa los punto 3 y 4 usando llamadas desde C

Ejercicio 2:

Imaginemos que tenemos una pequeña matriz de pesos para una red neuronal (por ejemplo, una capa fully connected) y queremos copiar esos valores a registros para su posterior uso.

Dado el siguiente arreglo en memoria:

.data

weights: .word 10, 20, 30, 40

Guía Práctica Programació nº 5 - RiscV

Implementa los punto 3 y 4 usando llamadas desde C

Ejercicio 3:

En una red, queremos identificar si alguna activación supera cierto umbral. Vamos a recorrer un arreglo de activaciones y verificar si alguna supera 50.

.data

activations: .word 45, 60, 22, 53

threshold: .word 50

result: .word 0

Gracias